

Лабораторная работа № 15: Исследование методов создания простого сервера с использованием фреймворка Koa.js.

Цель работы: Сформировать умение анализировать методы создания простого сервера с использованием фреймворка Koa.js.

Задание 1 (Базовый уровень)

Создание простого HTTP-сервера

Описание:

Необходимо создать базовый веб-сервер на Koa.js, который при обращении к корневому адресу будет отображать HTML-страницу с информацией о лабораторной работе и группе студента.

Как это должно работать:

1. Запускается сервер на порту 3000
2. При открытии в браузере адреса `http://localhost:3000` пользователь видит HTML-страницу
3. На странице отображается:
 - Заголовок "Лабораторная работа №15"
 - Информация о группе (ББМО-01-23)
 - Текущая дата и время
 - Приветственное сообщение

Пример работы:

- Запрос: GET `http://localhost:3000/`
 - Ответ: HTML-страница с оформленным содержимым
 - Сервер выводит в консоль сообщение о запуске
-

Задание 2 (Средний уровень)

Обработка различных HTTP-методов и маршрутов

Описание:

Необходимо расширить функциональность сервера для работы с

пользователями через REST API. Сервер должен обрабатывать запросы на создание, чтение, обновление и удаление пользователей.

Как это должно работать:

1. GET `/api/users` — получение списка всех пользователей
 - Отправляется GET-запрос на `/api/users`
 - Сервер возвращает JSON-массив с объектами пользователей
 - Пример ответа: `[{"id":1, "name":"Иванов Иван", "group":"ББМ0-01-23"}, ...]`
2. POST `/api/users` — добавление нового пользователя
 - Отправляется POST-запрос на `/api/users` с JSON-телом
 - Тело запроса: `{"name":"Сидоров Сидор", "group":"ББМ0-01-23"}`
 - Сервер добавляет пользователя с уникальным ID
 - Возвращает созданного пользователя с присвоенным ID
3. PUT `/api/users/1` — обновление пользователя
 - Отправляется PUT-запрос на `/api/users/1` (где 1 — ID пользователя)
 - Тело запроса: `{"name":"Петров Петр", "group":"ББМ0-02-23"}`
 - Сервер находит пользователя по ID и обновляет его данные
 - Возвращает обновленного пользователя
4. DELETE `/api/users/1` — удаление пользователя
 - Отправляется DELETE-запрос на `/api/users/1`
 - Сервер удаляет пользователя с указанным ID
 - Возвращает сообщение об успешном удалении

Обработка ошибок:

- При попытке получить/обновить/удалить несуществующего пользователя — возвращается ошибка 404
 - При невалидных данных в POST/PUT — возвращается ошибка 400
-

Задание 3 (Продвинутый уровень)

Middleware и обработка ошибок

Описание:

Реализовать систему промежуточных обработчиков (middleware) для логирования, обработки ошибок и авторизации запросов.

Как это должно работать:

1. Middleware логирования:

- При каждом запросе автоматически фиксируется:
 - Время запроса (начало и окончание)
 - HTTP-метод (GET, POST, PUT, DELETE)
 - Путь запроса
 - Время выполнения запроса в миллисекундах
- Вся информация выводится в консоль в формате:
- `text`
- `[2024-01-15 14:30:25] GET /users - 15ms`

2. Middleware обработки ошибок:

- Любые ошибки, возникающие в процессе обработки запроса, перехватываются
- Пользователь получает JSON-ответ с описанием ошибки
- Статус-код ошибки соответствует типу ошибки
- Пример ответа при ошибке: `{"error": "Внутренняя ошибка сервера", "status": 500}`

3. Middleware авторизации:

- Запросы к защищенным маршрутам проверяются на наличие заголовка `Authorization`
- Если заголовок отсутствует — возвращается ошибка 401
- Если заголовок присутствует — запрос обрабатывается дальше
- Для тестирования создается маршрут `/protected`, доступный только с авторизацией

4. Тестирование ошибок:

- Создается маршрут `/error`, который при обращении выбрасывает исключение
- Это позволяет проверить работу обработчика ошибок

Задание 4 (Сложный уровень)

Работа с параметрами URL и query-параметрами

Описание:

Разработать полноценное API для управления списком студентов с возможностью фильтрации, получения конкретного студента и выполнения всех CRUD-операций.

Как это должно работать:

1. GET `/students` — получение всех студентов

- Возвращает полный список студентов в формате JSON
- Каждый студент содержит: id, имя, группу, курс
- 2. GET `/students?group=ББМО-01-23` — фильтрация по группе
 - При добавлении query-параметра `group` возвращаются только студенты указанной группы
 - Пример: `/students?group=ББМО-01-23` вернет только студентов этой группы
 - Если группа не найдена — возвращается пустой массив
- 3. GET `/students/1` — получение конкретного студента
 - Возвращает студента с указанным ID
 - Если студент не найден — ошибка 404
- 4. POST `/students` — добавление студента
 - Тело запроса: `{"name": "Анна", "group": "ББМО-01-23", "course": 2}`
 - Добавляется новый студент с автоматической генерацией ID
 - Возвращается созданный студент
- 5. PUT `/students/1` — обновление студента
 - Обновляет данные студента по ID
 - Тело запроса содержит обновляемые поля
 - Возвращает обновленного студента
- 6. DELETE `/students/1` — удаление студента
 - Удаляет студента по ID
 - Возвращает подтверждение удаления

Дополнительная логика:

- При создании студента проверяется наличие всех обязательных полей
- Если поле `group` содержит номер группы (например, "ББМО-01-23"), сервер может выполнять вычисления на основе номера группы
- Валидация данных перед сохранением

Задание 5 (Экспертный уровень)

Полноценное REST API с генерацией данных по номеру группы

Описание:

Создать полнофункциональное REST API для управления студентами с автоматической генерацией тестовых данных, пагинацией, сортировкой, поиском и валидацией.

Как это должно работать:

1. Генерация данных:
 - При запуске сервера автоматически создается 50 студентов
 - Данные генерируются на основе номера группы (ББМО-01-23)
 - Генерируются реалистичные имена и фамилии
 - Студенты распределяются по разным группам
 - Каждому студенту присваивается курс (1-4)
 2. GET /students с пагинацией:
 - Поддерживаются параметры:
 - `limit` — количество записей на странице (по умолчанию 10)
 - `offset` — смещение (сколько записей пропустить)
 - Пример: `/students?limit=5&offset=10` — вернет 5 записей, начиная с 11-й
 - В ответе возвращается общее количество записей
 3. Сортировка:
 - Параметр `sort` позволяет сортировать по полям:
 - `sort=name` — сортировка по имени (A-Z)
 - `sort=-name` — сортировка по имени (Z-A)
 - `sort=course` — сортировка по курсу
 - Можно комбинировать: `/students?sort=course&limit=10`
 4. Поиск по имени:
 - Параметр `search` для поиска по части имени
 - Пример: `/students?search=Алек` — найдет всех студентов с именами, содержащими "Алек"
 - Поиск регистронезависимый
 5. CRUD операции:
 - Полный набор: GET (все/один), POST, PUT, DELETE
 - Валидация входных данных
 - Обработка ошибок
 6. Пример работы:
 - GET `/students?limit=10&offset=0&sort=name&search=Ан`
 - Ответ содержит: первые 10 студентов, у которых в имени есть "Ан", отсортированных по имени, и общее количество найденных
-

Фиксация изменений

Добавить файлы в Git

Создать коммит

Отправить в удалённый репозиторий

Создать Pull Request